Filter Data - WHERE

In the previous chapter we saw how to `SELECT` data from the database.

We did it using pure **SQL** and using **SQLModel**.

But we always got all the rows, the whole table:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

In most of the cases we will want to get only one row, or only a group of rows.

We will see how to do that now, to filter data and get only the rows **where** a condition is true.

Continue From Previous Code

We'll continue with the same examples we have been using in the previous chapters to create and select data.

And now we will update `select_heroes()` to filter the data.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

If you already executed the previous examples and have a database with data, **remove the database file** before running each example, that way you won't have duplicate data and you will be able to get the same results.

Filter Data with SQL

Let's check first how to filter data with **SQL** using the `WHERE` keyword.

```sql
SELECT id, name, secret_name, age
FROM hero
WHERE name = "Deadpond"
```

The first part means the same as before:

> Hey SQL database 👋, please go and `SELECT` some data for me.
>
> I'll first tell you the columns I want:
>
> - `id`
> - `name`
> - `secret_name`
> - `age`
>
> And I want you to get them `FROM` the table called `"hero"`.

Then the `WHERE` keyword adds the following:

> So, SQL database, I already told you what columns to `SELECT` and where to select them `FROM`. But I don't want you to bring me all the rows, I only want the rows `WHERE` the `name` column has a value of `"Deadpond"`.

Then the database will bring a table like this:

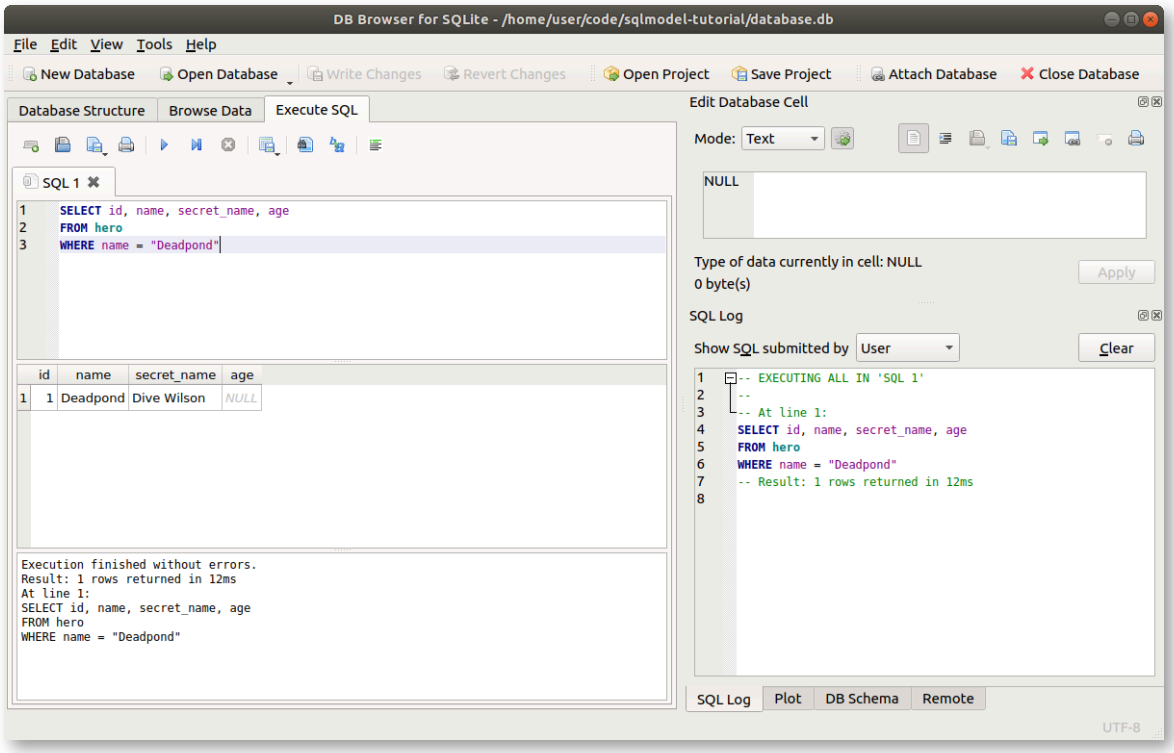
id	name	secret_name	age
1	Deadpond	Dive Wilson	null

Tip

Even if the result is only one row, the database always returns a **table**.

In this case, a table with only one row.

You can try that out in **DB Browser for SQLite**:



`WHERE` and `FROM` are "clauses"

These additional keywords with some sections like `WHERE` and `FROM` that go after `SELECT` (or others) have a technical name, they are called **clauses**.

There are others clauses too, with their own SQL keywords.

I won't use the term **clause** too much here, but it's good for you to know it as it will probably show up in other tutorials you could study later. 🤓

`SELECT` and `WHERE`

Here's a quick tip that helps me think about it.

- **`SELECT`** is used to tell the SQL database what **columns** to return.
- **`WHERE`** is used to tell the SQL database what **rows** to return.

The size of the table in the two dimensions depend mostly on those two keywords.

`SELECT` Land

If the table has too many or too few **columns**, that's changed in the **`SELECT`** part.

Starting with some table:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

...and changing the number of **columns**:

name
Deadpond
Spider-Boy
Rusty-Man

...is all `SELECT` land.

`WHERE` Land

If the table has too many or too few **rows**, that's changed in the **`WHERE`** part.

Starting with some table:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null

2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

...and changing the number of **rows**:

id	name	secret_name	age
2	Spider-Boy	Pedro Parqueador	null

...is all `WHERE` land.

Review `SELECT` with SQLModel

Let's review some of the code we used to read data with **SQLModel**.

We care specially about the **select** statement:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Filter Rows Using `WHERE` with SQLModel

Now, the same way that we add `WHERE` to a SQL statement to filter rows, we can add a `.where()` to a **SQLModel** `select()` statement to filter rows, which will filter the objects returned:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
```

```
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

It's a very small change, but it's packed of details. Let's explore them.

`select()` Objects

The object returned by `select(Hero)` is a special type of object with some methods.

One of those methods is `.where()` used to (unsurprisingly) add a `WHERE` to the SQL statement in that **select** object.

There are other methods that we will explore later. 💡

Most of these methods return the same object again after modifying it.

So we could call one after the other:

```
statement = select(Hero).where(Hero.name == "Deadpond").where(Hero.age == 48)
```

Calling `.where()`

Now, this `.where()` method is special and very powerful. It is tightly integrated with **SQLModel** (actually SQLAlchemy) to let you use very familiar Python syntax and code.

Notice that we didn't call it with a single equal (`=`) sign, and with something like:

```
# Not supported 🚨
select(Hero).where(name="Deadpond")
```

That would have been shorter, of course, but it would have been much more error prone and limited. I'll show you why in a bit.

Instead, we used two `==`:

```
select(Hero).where(Hero.name == "Deadpond")
```

So, what's happening there?

`.where()` and Expressions

In the example above we are using two equal signs (`==`). That's called the "**equality operator**".

> **Tip**
>
> An **operator** is just a symbol that is put beside one value or in the middle of two values to do something with them.
>
> `==` is called the **equality** operator because it checks if two things are **equal**.

When writing Python, if you write something using this equality operator (`==`) like:

```
some_name == "Deadpond"
```

...that's called an equality "**comparison**", and it normally results in a value of:

```
True
```

...or

```
False
```

> **Tip**
>
> `<`, `>`, `==`, `>=`, `<=`, and `!=` are all **operators** used for **comparisons**.

But SQLAlchemy adds some magic to the columns/fields in a **model class** to make those Python comparisons have super powers.

So, if you write something like:

```
Hero.name == "Deadpond"
```

...that doesn't result in a value of `True` or `False`. 🤯

Instead, it results in a special type of object. If you tried that in an interactive Python session, you'd see something like:

```
>>> Hero.name == "Deadpond"
<sqlalchemy.sql.elements.BinaryExpression object at 0x7f4aec0d6c90>
```

So, that result value is an **expression** object. 💡

And `.where()` takes one (or more) of these **expression** objects to update the SQL statement.

Model Class Attributes, Expressions, and Instances

Now, let's stop for a second to make a clear distinction that is very important and easy to miss.

Model class attributes for each of the columns/fields are special and can be used for expressions.

But that's only for the **model class attributes**. 🚨

Instance attributes behave like normal Python values. ✅

So, using the class (`Hero`, with capital `H`) in a Python comparison:

```
Hero.name == "Deadpond"
```

...results in one of those **expression** objects to be used with `.where()`:

```
<sqlalchemy.sql.elements.BinaryExpression object at 0x7f4aec0d6c90>
```

But if you take an instance:

```
some_hero = Hero(name="Deadpond", secret_name="Dive Wilson")
```

...and use it in a comparison:

```
some_hero.name == "Deadpond"
```

...that results in a Python value of:

```
True
```

...or if it was a different object with a different name, it could have been:

```
False
```

The difference is that one is using the **model class**, the other is using an **instance**.

Class or Instance

It's quite probable that you will end up having some variable `hero` (with lowercase `h`) like:

```
hero = Hero(name="Deadpond", secret_name="Dive Wilson")
```

And now the class is `Hero` (with capital `H`) and the instance is `hero` (with a lowercase `h`).

So now you have `Hero.name` and `hero.name` that look very similar, but are two different things:

```
>>> Hero.name == "Deadpond"
<sqlalchemy.sql.elements.BinaryExpression object at 0x7f4aec0d6c90>

>>> hero.name == "Deadpond"
True
```

It's just something to pay attention to. 🤓

But after understanding that difference between classes and instances it can feel natural, and you can do very powerful things. 🚀

For example, as `hero.name` works like a `str` and `Hero.name` works like a special object for comparisons, you could write some code like:

```
select(Hero).where(Hero.name == hero.name)
```

That would mean:

> Hey SQL Database 👋, please `SELECT` all the columns
>
> `FROM` the table for the model class `Hero` (the table `"hero"`)
>
> `WHERE` the column `"name"` is equal to the name of this hero instance I have here: `hero.name` (in the example above, the value `"Deadpond"`).

`.where()` and Expressions Instead of Keyword Arguments

Now, let me tell you why I think that for this use case of interacting with SQL databases it's better to have these expressions:

```
# Expression ✨
select(Hero).where(Hero.name == "Deadpond")
```

...instead of keyword arguments like this:

```
# Not supported, keyword argument 🚨
select(Hero).where(name="Deadpond")
```

Of course, the keyword arguments would have been a bit shorter.

But with the **expressions** your editor can help you a lot with autocompletion and inline error checks. ✨

Let me give you an example. Let's imagine that keyword arguments were supported in SQLModel and you wanted to filter using the secret identity of Spider-Boy.

You could write:

```
# Don't copy this 🚨
select(Hero).where(secret_identity="Pedro Parqueador")
```

The editor would see the code, and because it doesn't have any information of which keyword arguments are allowed and which not, it would have no way to help you **detect the error**.

Maybe your code could even run and seem like it's all fine, and then some months later you would be wondering why your app *never finds rows* although you were sure that there was one `"Pedro Parqueador"`. 😱

And maybe finally you would realize that we wrote the code using `secret_identity` which is not a column in the table. We should have written `secret_name` instead.

Now, with the expressions, your editor would show you an error right away if you tried this:

```
# Expression ✨
select(Hero).where(Hero.secret_identity == "Pedro Parqueador")
```

Even better, it would autocomplete the correct one for you, to get:

```
select(Hero).where(Hero.secret_name == "Pedro Parqueador")
```

I think that alone, having better editor support, autocompletion, and inline errors, is enough to make it worth having expressions instead of keyword arguments. ✨

> **Tip**
>
> **Expressions** also provide more features for other types of comparisons, shown down below. 👇

Exec the Statement

Now that we know how `.where()` works, let's finish the code.

It's actually the same as in previous chapters for selecting data:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

We take that statement, that now includes a `WHERE`, and we `exec()` it to get the results.

And in this case the results will be just one:

```bash
```

Tip

The `results` object is an iterable to be used in a `for` loop.

Even if we got only one row, we iterate over that `results` object. Just as if it was a list of one element.

We'll see other ways to get the data later.

Other Comparisons

Here's another great advantage of these special **expressions** passed to `.where()`.

Above, we have been using an "equality" comparison (using `==`), only checking if two things are the same value.

But we can use other standard Python comparisons. ✨

Not Equal

We could get the rows where a column is **not** equal to a value using `!=`:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name != "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name != "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

That would output:

```
secret_name='Pedro Parqueador' age=None id=2 name='Spider-Boy'
secret_name='Tommy Sharp' age=48 id=3 name='Rusty-Man'
```

Pause to Add Data

Let's update the function `create_heroes()` and add some more rows to make the next comparison examples clearer:

Python 3.10+

```python
# Code above omitted 👆
```

```python
def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age > 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Now that we have several heroes with different ages, it's gonna be more obvious what the next comparisons do.

More Than

Now let's use `>` to get the rows where a column is **more than** a value:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age > 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age > 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

That would output:

```
age=48 id=3 name='Rusty-Man' secret_name='Tommy Sharp'
age=36 id=6 name='Dr. Weird' secret_name='Steve Weird'
age=93 id=7 name='Captain North America' secret_name='Esteban Rogelios'
```

Tip

Notice that it didn't select `Black Lion`, because the age is not *strictly* greater than `35`.

More Than or Equal

Let's do that again, but with `>=` to get the rows where a column is **more than or equal** to a value:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Because we are using `>=`, the age `35` will be included in the output:

```
age=48 id=3 name='Rusty-Man' secret_name='Tommy Sharp'
age=35 id=5 name='Black Lion' secret_name='Trevor Challa'
age=36 id=6 name='Dr. Weird' secret_name='Steve Weird'
age=93 id=7 name='Captain North America' secret_name='Esteban Rogelios'
```

Tip

This time we got `Black Lion` too because although the age is not *strictly* greater than `35` it is *equal* to `35`.

Less Than

Similarly, we can use `<` to get the rows where a column is **less than** a value:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And we get the younger one with an age in the database:

```
age=32 id=4 name='Tarantula' secret_name='Natalia Roman-on'
```

Tip

We could imagine that **Spider-Boy** is even **younger**. But because we don't know the age, it is `NULL` in the database (`None` in Python), it doesn't match any of these age comparisons with numbers.

Less Than or Equal

We can also use `<=` to get the rows where a column is **less than or equal** to a value:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And we get the younger ones, `35` and below:

```
age=32 id=4 name='Tarantula' secret_name='Natalia Roman-on'
age=35 id=5 name='Black Lion' secret_name='Trevor Challa'
```

Tip

We get `Black Lion` here too because although the age is not *strictly* less than `35` it is *equal* to `35`.

In

Finally, we can use `in_` to get the rows where a column is a member of a collection of values:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.name).in_(["Deadpond", "Ratman"]))
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, col, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.name).in_(["Deadpond", "Ratman"]))
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

🤓 **Other versions and variants**

Python 3.9+

```python
from typing import Optional

from sqlmodel import Field, Session, SQLModel, col, create_engine, select


class Hero(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: Optional[int] = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.name).in_(["Deadpond", "Ratman"]))
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

In this case, we match `Deadpond` since it's part of the collections of names. We don't have any hero called `Ratman`, so it does not match any hero.

> **Tip**
>
> You need to wrap your attribute with `col()` to use `in_`. You can read more about it in the Type annotations and errors section.

Benefits of Expressions

Here's a good moment to see that being able to use these pure Python expressions instead of keyword arguments can help a lot. ✨

We can use the same standard Python comparison operators like `<`, `<=`, `>`, `>=`, `==`, etc.

Multiple `.where()`

Because `.where()` returns the same special select object back, we can add more `.where()` calls to it:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35).where(Hero.age < 40)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35).where(Hero.age < 40)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This will select the rows `WHERE` the `age` is **greater than or equal** to `35`, `AND` also the `age` is **less than** `40`.

The equivalent SQL would be:

```sql
SELECT id, name, secret_name, age
FROM hero
WHERE age >= 35 AND age < 40
```

This uses `AND` to put both comparisons together.

We can then run it to see the output from the program:

```bash
```

`.where()` With Multiple Expressions

As an alternative to using multiple `.where()` we can also pass several expressions to a single `.where()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35, Hero.age < 40)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age >= 35, Hero.age < 40)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This is the same as the above, and will result in the same output with the two heroes:

```
age=35 id=5 name='Black Lion' secret_name='Trevor Challa'
age=36 id=6 name='Dr. Weird' secret_name='Steve Weird'
```

`.where()` With Multiple Expressions Using `OR`

These last examples use `where()` with multiple expressions. And then those are combined in the final SQL using `AND`, which means that *all* of the expressions must be true in a row for it to be included in the results.

But we can also combine expressions using `OR`. Which means that **any** (but not necessarily all) of the expressions should be true in a row for it to be included.

To do it, you can import `or_`:

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine, or_, select

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, or_, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(or_(Hero.age <= 35, Hero.age > 90))
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And then pass both expressions to `or_()` and put it inside `.where()`.

For example, here we select the heroes that are the youngest OR the oldest:

Python 3.10+

```python
# Code above omitted 👆
```

```python
def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(or_(Hero.age <= 35, Hero.age > 90))
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, or_, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(or_(Hero.age <= 35, Hero.age > 90))
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

When we run it, this generates the output:

```bash
```

Type Annotations and Errors

There's a chance that your editor gives you an error when using these comparisons, like:

```
Hero.age > 35
```

It would be an error telling you that

> `Hero.age` is potentially `None`, and you cannot compare `None` with `>`

This is because as we are using pure and plain Python annotations for the fields, `age` is indeed annotated as `int | None`.

By using this simple and standard Python type annotations we get the benefit of the extra simplicity and the inline error checks when creating or using instances. ✨

And when we use these special **class attributes** in a `.where()`, during execution of the program, the special class attribute will know that the comparison only applies for the values that are not `NULL` in the database, and it will work correctly.

But the editor doesn't know that it's a special **class attribute**, so it tries to help us preventing an error (that in this case is a false alarm).

Nevertheless, we can easily fix. 🎉

We can tell the editor that this class attribute is actually a special **SQLModel** column (instead of an instance attribute with a normal value).

To do that, we can import `col()` (as short for "column"):

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, col, create_engine, select
```

```
# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, col, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.age) >= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And then put the **class attribute** inside `col()` when using it in a `.where()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.age) >= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, col, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(col(Hero.age) >= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

So, now the comparison is not:

```python
Hero.age > 35
```

...but:

```python
col(Hero.age) > 35
```

And with that the editor knows this code is actually fine, because this is a special **SQLModel** column.

> **Tip**
>
> That `col()` will come handy later, giving autocompletion to several other things we can do with these special **class attributes** for columns.
>
> But we'll get there later.

Recap

You can use `.where()` with powerful expressions using **SQLModel** columns (the special class attributes) to filter the rows that you want. 🚀

Up to now, the database would have been **looking through each one of the records** (rows) to find the ones that match what you want. If you have thousands or millions of records, this could be very **slow**. 😱

In the next section I'll tell you how to add **indexes** to the database, this is what will make the queries **very efficient**. 😎